

حلول ورقة العمل 05: التعارضات والنشر

Solutions · Worksheet 05 — Conflicts & Deployment

معيار النجاح / Pass mark
70 نقطةأدلة إلزامية / Mandatory evidence
9 بنودزمن التصحيح المقترح / Marking time
10-12 دقيقة (تقرير + نشر)التقييم الكلي / Total
100 نقطة

نسخة المدرب — نماذج الحلول والتقييم / Instructor copy — do not distribute

فرق مهم في هذه الورقة A key difference in this worksheet

الورقة 05 قياس قدرة على حل مشكلة تحت ضغط، لا حفظ أوامر. لذلك نجحت المهمة حتى لو اختلف الحل عن هذا النموذج، بشرط أن يكون: (1) التاريخ سليماً بلا حذف، (2) عمل الزملاء محفوظاً، (3) الفحص التلقائي ينجح. علم طلابك أن هناك حلولاً متعددة صحيحة – وأن الخطأ الحقيقي الوحيد هو إتلاف عمل غيرك أو إخفاء علامات تعارض في المشروع.

EN

This worksheet measures problem-solving under pressure, not recall. Any solution is valid if the history is intact, teammates' work preserved, and validation passes.

جدول التقييم المُجمَع Marking rubric

# المهمة	النقاط معيار القبول	حساسية الأمان
1 تعارض سطر واحد	20	تعارض مقصود + حل يُبقي المعلومات لا علامات باقية
2 تعارض ملف مُوَلَّد	20	حلّ بإعادة التوليد لا يدوياً
3 rebase و force-with-lease	15	فهم الفرق + دفع آمن
4 الفحوص الستة	15	6 فحوص منقّذة بمخرجات
5 النشر على Pages	15	رابط يعمل + كل البطاقات
6 PR نهائي + تقرير	15	PR مكتمل + تقرير بأرقام
– الأسئلة التحليلية	15	4 أسئلة × 4
		مراجعة زميل موثقة
		يقيس الفهم العميق

إجابات المهام Task solutions

1 تعارض سطر واحد

Single-line conflict

★ 20 نقطة

◆ إتقان

الشكل الصحيح للتعارض الذي يجب أن ينتجه الطالب:

```
<<<<<< HEAD
شعار الفريق: «نتعلّم معاً، ونرفع معاً» - أعضاء الفريق: سارة
=====
شعار الفريق: «نتعلّم معاً، ونرفع معاً» - أعضاء الفريق: عمر
>>>>>> docs/omar-slogan
```

الحل النموذجي:

شعار الفريق: «نتعلّم معاً، ونرفع معاً» - أعضاء الفريق: سارة، عمر

```
$ git add sandbox/team-slogan.txt
$ git commit --no-edit
[main d34a3f6] Merge pull request #22 from omar/slogan
$ git diff --check          # لا مخرجات
```

سَلِّم تقييم الحل (20 نقطة):

النقاط	العنصر
5	أنتج تعارضاً حقيقياً (مخرجات <code>CONFLICT (content)</code>)
4	الصق مقطع العلامات كما ظهر على شاشته
5	السطر النهائي التنظيف بلا أي علامة
4	<code>git add</code> ثم <code>git commit --no-edit</code> ثم نجاح الإكمال
2	<code>git diff --check</code> بلا مخرجات (تحقق ذاتي)

أخطاء متوقعة وخصوصياتها:

الخطأ	النتيجة في المشروع	الخصم
اختار نسخه فقط وحذف اسم الزميل	ضياع مساهمة زميل	8- (خلل تعاوني)
بقيت <code>HEAD <<<<<<</code> داخل الملف	المشروع مكسور	10- إذا لم يلاحظها، 3- إن أصلحها بعد <code>diff --check</code>
<code>git add</code> منسي ثم <code>git commit</code>	<code>nothing to commit</code>	لا خصم إن فهم السبب وسأل
استخدم <code>git merge --abort</code> ثم أعاد المحاولة	سلوك صحيح ومحمود	لا خصم
حلّل التعارض بتحرير النص في VS Code	الأسهل والأفضل	لا خصم، بل شجّع

فحص إلزامي بعد كل حل تعارض (لا تُسَلِّم ورقة بلا هذا الفحص) Mandatory post-resolution check

```
$ grep -rn "<<<<<<|>>>>>>|=====" --include="*.json" --include="*.txt" --include="*.md" .
$ git diff --check
```

إن ظهر أي ملف فيه علامات، فالمشروع **مكسور** ويجب ألا يُقبل التسليم.

2 تعارض الملف القولد

Generated-file conflict

★ 20 نقطة

◆ إتقان

الحل الصحيح الوحيد (وهو مقياس فهم الورقة):

```
$ node tools/build-index.mjs
[build-index] ✓ بطاقة 5. تم توليد الفهرس:
  • example-student • leen-docs • omar-qa • sara-design • you-instructor

$ git add data/members-index.json data/members-bundle.js
$ git rebase --continue
[detached HEAD 9a8b7c6] feat(members): add card for omar-qa
Successfully rebased and updated refs/heads/feature/omar-qa-card.
```

أخطاء متوقعة وخصوماتها:

الدرجة	التقييم	الحل الذي كتبه الطالب
20	الحل الاحترافي	أعد التوليد ثم <code>add</code> ثم <code>rebase --continue</code>
12	ينجح الآن ويفشل لاحقاً – ناقص المفهوم	حلّ التعارض يدوياً بكتابة السطرين معاً (<code>"leen-docs.json", "omar-qa.json",</code>
14	مقبول عدا خسارة تجربة	ألغى <code>rebase</code> كاملاً (<code>--abort</code>) وأعاد العمل من الصفر على فرع جديد
10-	يتسبب باختفاء بطاقة زميل من الموقع	اختار نسخته وحذف مدخل الزميل
3-	خطأ إجرائي	لم يفرّق بين <code>rebase --continue</code> و <code>merge --continue</code>
تعليق إلزامي في التقييم	يكرر الخطأ	لم يفهم لماذا الحدث (لماذا هذا الملف مشترك)

السؤال الذي يفرز الفهم *The separating question*

اسأل: «لماذا كان حلّ `members-index.json` يدوياً صحيحاً تقنياً وخطأً منهجياً؟» الإجابة الكاملة: «لأن الملف **نتيجة** لا مصدر – تولّده الأداة من محتوى المجلد. وأي تعديل يدوي سيُطمس عند التوليد التالي، وقد يبقى مدخل زميل ناقصاً بلا أن يلاحظ أحد.» من أجاب بهذا المنطق فهو جاهز لقيادة فريق.

3 rebase والدفع الآمن

Rebase and safe push

★ 15 نقطة

◆ فهم أمني

الإجابة النموذجية:

```
$ git push
! [rejected]          feature/omar-qa-card → feature/omar-qa-card (non-fast-forward)
...
$ git push --force-with-lease origin feature/omar-qa-card
+ 1af7c9b...9a8b7c6 feature/omar-qa-card → feature/omar-qa-card (forced update)
```

معيار القبول (15 نقطة): | العنصر | النقاط | |---|---| | مخرجات رفض الدفع العادي (`non-fast-forward`) | 4 | | نجاح | 4 | | صحيح الصيغة | 4 | | شرح أن الآلية تقارن بحالة الفرع البعيد المعروفة محلياً وترفض إن تغيّرت | 5 | | ذكر أن الصلاحية للفرع الشخصي فقط وليست لـ `main` | 2 |

أخطاء متوقعة:

الخطأ	الخطورة	الخصم
استخدم <code>git push --force</code>	خطر فقدان عمل زميل -8 + إعادة تدريب	

الخطأ

الخطورة

الخصم

- نقذ `--force` على `main` مخالفة جسيمة
- قال «الفرق أن with-lease أطول» لم يفهم
- استخدم `git push -f` على فرع زميل سلوك مرفوض
- لم يلاحظ `ahead 1, behind 3` لا يعرف قراءة الحالة

درس أمني أوسع — اربطه بالمشروع Wider security lesson

هذه أول مرة يرى فيها الطالب أن «القوة» في Git قد تدمر عمل غيره — وهو نفس المنطق في `git clean -fd` و `git reset --hard` و `git branch -D`. اجعل القاعدة صريحة على السبورة: «أوامر القوة المدمرة: للفرع الشخصي فقط، وبعد التحقق، ومع نسخة احتياطية من العمل.»

4 الفحوص الستة قبل المراجعة

Six pre-review checks

★ 15 نقطة منهج

الإجابات النموذجية:

الأمر

النتيجة الصحيحة

- `node tests/validate-members.mjs` خطأ ، 0 تحذير 0
 - `git diff --check` لا مخرجات
 - `git diff --stat origin/main...HEAD` ملفات البطاقة + الفهرس + ال bundle فقط
 - `git log --oneline origin/main..HEAD` التزام (أو التزامات) بصيغة صحيحة
 - `git status -sb` لا `behind` ، ولا `diverged`
 - `git diff origin/main...HEAD | grep -iE "token|password"` لا نتائج
- معيار القبول:** 2.5 نقطة لكل فحص منقذ بمخرجات أو بنتيجة مذكورة. الفحص 6 هو **الأهم أمنياً**؛ من تركه فارغاً بلا سبب ⇒ لا تُمنح نقاطه حتى لو نجحت الفحوص الأخرى، لأن الدرس الأمني مقصود ولا يمكن تقييم غيره مكانه.
- أخطاء متوقعة:** من كتب `git diff main..HEAD` (نقطتان) بدل `...` (ثلاث) فيظهر الفرق معاكساً ← وجّهه؛ لا تخصم في بند الفحوص لكن اذكرها ملاحظة. من نقذ `git fetch` أولاً قبل المقارنة ← ممتاز، أضف نقطة تقدير (bonus) وسجلها.

استثمار تعليمي: اجعل الفحص عادة Turn checks into habit

اطلب في كل ورقة قادمة (ولو بنداً واحداً) تنفيذ الفحص `git diff --check` قبل أي التزام. الطلاب الذين يبنون عادة «افحص قبل أن ترفع» يعودون من أي جلسة عمل بلا مفاجآت — وهذه المهارة تنقلهم من «متعلمين» إلى «موثوقين».

5 النشر على GitHub Pages

Deploy to GitHub Pages

◆ تنفيذ ★ 15 نقطة

الإجابة النموذجية: رابط من الصيغة `https://<org>.github.io/class-team-hub/` يعمل، وتظهر فيه كل بطاقات الفريق.

معيار القبول: 10 نقاط لربط يعمل ويفتح الشاشة الرئيسية؛ 5 نقاط لظهور بطاقة الطالب نفسه بين البطاقات (فلا تظهر صورة شاشة للصفحة الرئيسية فقط).

أخطاء متوقعة ومصفوفة تشخيص سريعة للمدرب:

العرض في الرابط	السبب الفعلي	علاج فوري في الجلسة
404 ثابت بعد 5 دقائق	لم يُفَعَّل Pages أو المسار خطأ	Save ثم Settings ← Pages ← Branch: <code>main</code> / Root
الصفحة تعمل بلا أي بطاقة	<code>fetch</code> فشل بسبب مسار غير مطابق للأحرف	افتح <code>F12 → Console</code> واقرأ الرسالة الحمراء
تظهر بطاقات قديمة (من ضمنها مثال)	لم يُحدَّث بالدمج <code>main</code>	ثم فحص الدمج على المستودع <code>git pull</code>
تظهر بطاقات لكن نقص عددها	بطاقة فيها JSON خطأ	على جهاز المدرب <code>node tests/validate-members.mjs</code>
الصفحة بيضاء تماماً	تعارض دمج وترك آثاراً	<pre>grep -rn "<<<<<<" --include="*.js" --include="*.html" .</pre>
يعمل لديه ولا يعمل للزائر	نسي <code>Commit</code> ل <code>members-index.json</code>	أعد التوليد والتزم وارفع

لحظة تحفيزية استخدمها *Use the motivational moment*

اجعل الطالب يفتح رابط الموقع المنشور ويُري بطاقته لوالده أو صديقه أو يرسلها في مجموعة الصف. هذه اللحظة («عمل منشور على الإنترنت باسمي، وفريق عمل عليه معي») أقوى محرّك لتعلم المزيد من عشر محاضرات. خُصّص لها دقيقتين في نهاية الجلسة.

6 طلب السحب النهائي والتقرير

Final PR and report

★ 15 نقطة

◆ إتقان

معيار القبول للطلب (7 نقاط): عنوان بصيغة صحيحة، الوصف يشمل `Closes #<issue>` ، طريقة الاختبار موثقة بمخرجات `validate-members` ، وطلب مراجعة من زميل، وتقديم ملف واحد فقط (`README.md`) غالباً بلا خلط مهام.

معيار القبول للتقرير (8 نقاط): أن تكون كل القيم حقيقية وقابلة للمطابقة مع المستودع. تحقّق ذكياً من سطرين فقط: (1) عدد البطاقات = عدد ملفات `data/members/*.json` ناقص قالب والمثال والمدرّب؛ (2) عدد الالتزامات في `main` يقارن بـ `git log --oneline main | wc -l` على النسخة المشتركة. التقرير المتفق مع الواقع = درجة كاملة.

أخطاء متوقعة: - أرقام بلا أي مقابل واقعي (توحي بالتقدير) ← -3 مع إعادة الطلب. - «أكبر تعارض واجهته» مكتوب نظرياً («التعارض يحدث عندما...») بدل تجربته ← -2. - لا ذكر لأي فرع فشل في دمج (يبدو الكمال غير الواقعي) ← لا خصم لكن شجّع الصديق. - PR نهائي يخلط ثلاثة ملفات مختلفة (`README + CSS` + بطاقة) ← -2 (خلط مهام).

- 1) بطاقة الطالب ظاهرة هناك؟ ☐
- 2) main (git status) على جهاز الطالب origin/main متساو مع ☐
- 3) node tests/validate-members.mjs → خطأ 0 ☐
- 4) عدد طلبات السحب التي أنشأها الطالب $3 \leq$ (له/مراجعة/أغلقت) ☐
- 5) لصق دليلين مختلفين لحل تعارضين (سطر + مُوَلَّد) في الورقة ☐
- 6) لا نتائج → . grep -rn "<<<<<<" لا آثار للعلامات ☐
- 7) بالمجات grep -rniE "token|password|secret" لا أسرار في المستودع ☐

إجابات الأسئلة التحليلية Analytical answers

س1 لماذا تعارض في نفس السطر فقط

Why same-line conflicts

★ 4 نقطة

تحليل

الجواب: Git يدمج على مستوى الأسطر والكتل، لا الملفات. عند تعديل أسطر متباعدة، كل تعديل يقع في «مقطع» مختلف من الملف فيدمجها تلقائياً بلا تدخل. أما تعديل نفس السطر من الجهتين فينتج **مقطعان متعارضان** لنفس الموضع، ولا يستطيع Git تخمين أيهما المقصود — فيتوقف ويسلم القرار للإنسان.

س2 معنى HEAD في merge مقابل rebase

HEAD in merge vs rebase

★ 4 نقطة

تحليل

الجواب: في تعارض **merge**: HEAD يمثل فرعك الحالي (المتلقّي)، والطرف الآخر هو الفرع الذي تحاول دمجه — أي «نسخي» مقابل «القادم». أما في تعارض **rebase**: يعمل Git على حالة HEAD منفصلة (detached)، وفيها HEAD يمثل الفرع الذي تُبنى عليه (أي main الجديد غالباً)، والطرف الآخر هو التزامك الذي يعيد تشكيله — أي أن المعنى **معكوس**. هذا هو سبب أخطاء الطلاب الشائعة: يظنون سطر HEAD نسختهم دائماً فيختارون الطرف الخطأ ويضيع عملهم.

س3 لو كان الفهرس يدوياً

If the index were manual

★ 4 نقطة

تحليل

الجواب: «كان كل طالب سيعدّل نفس الملف في نفس الموضع تقريباً، فيقع تعارض في كل طلب سحب تقريراً، ويحتاج كل دمج إلى تدخل بشري. ومع 20 طالباً كان تاريخ main سيصبح سلسلة تعارضات يُحلّ بعضها بالخطأ (حذف مدخل زميل، أو تكراره، أو خروج ترتيب الملف)، وتزداد أخطاء JSON ويصبح رفع البطاقة عملية مرهقة بدل أن تكون بسيطة.» **الخلاصة:** لكي يعمل الفريق، اجعل الملفات المشتركة أقل ما يمكن، واستخدم ملفات فردية + توليد آلي.

س4 تعديل مقترح على سير العمل

Workflow improvement proposal

★ 3 نقطة

قيادة

إجابات ممتازة مقبولة: - تفعيل **Code Owners** بحيث لا يُدمج تعديل ملف مشترك إلا بمراجعة قائد الفريق. - توحيد الاجتماع اليومي القصير (Stand-up) لمزامنة الفروع قبل بدء العمل. - قاعدة: «لا فرع يعيش أكثر من يومين» - لتقليل التعارضات. - إضافة أداة فحص تلقائي (GitHub Actions) تشغل `validate-members` على كل طلب سحب، فيرفض الطلب آلياً قبل أن يصل للمراجعة البشرية. - تخصيص ملف واحد لكل طالب دائماً، وأي تعديل جماعي يُوضع في ملف جديد لتفادي التعارض.

مقياس الدرجة: نقطتان لفكرة عملية قابلة للتنفيذ في المشروع + نقطة لربطها بتقليل التعارضات تحديداً. الفكرة العامة («نتعاون أكثر») بلا آلية ⇒ نقطة واحدة.

ملخص التصحيح والتقييم النهائي *Final marking summary*

سجل الدرجات النهائي / *Final grade sheet*

الاسم م1/ 20 م2/ 20 م3/ 15 م4/ 15 م5/ 15 م6/ 15 أسئلة/ 15 المجموع/ 100 رابط الموقع حالة الإتمام

معايير الشهادة النهائية (اقتراح) *Suggested certification criteria*

المستوى	الشرط
إتمام أساسي	+70 في المجموع الكلي للورق الخمس، وبطاقة مدموجة على <code>main</code>
إتمام متقدم	+85 في المجموع، مع 3 طلبات سحب (له/راجع/أغلق بالمراجعة) ودليل حل تعارضين
قائد فريق	إتمام متقدم + مراجعات مكتوبة عالية الجودة لثلاثة زملاء + تقرير مشروع بأرقام دقيقة، وحل تعارض زميل عملياً أمام
مدرّب	المجموعة

بعد الورشة — ماذا ترسل للطلاب؟ *Post-workshop hand-off*

1. رابط المستودع المشترك + رابط الموقع المنشور (للعرض في يوم المدرسة المفتوح).
2. ملف `docs/git-cheatsheet.md` من داخل المشروع نفسه (مرجعهم بعد انتهاء الورشة).
3. مهمة إثراء اختيارية: أضف ميزة واحدة للموقع (بحث محسن، فترة بالوسوم، وضع ليلي) عبر فرع + طلب سحب — لتستمر المهارة بعد البرنامج.